

User Service & Property Management System

(REST API Using FastAPI And PostgreSQL)

1. Project Overview

The **User Service and Property Management System** is a web-based application developed to manage users, tenants, and properties in a structured relational database. The system allows an administrator or authorized user to: Create users, View users, Update users, Delete users, Create tenants, View tenants, Update tenants, Delete tenants, Create properties, View properties, Update properties, Delete properties, Assign users to tenants, Assign properties to users, Assign properties to tenants, View how many properties belong to a user, View how many users belong to a tenant, View how many properties belong to a tenant, View the properties belonging to a user or tenant, Authenticate users using username/password, Protect APIs using JWT authentication, Control access using user roles.

2. Objectives

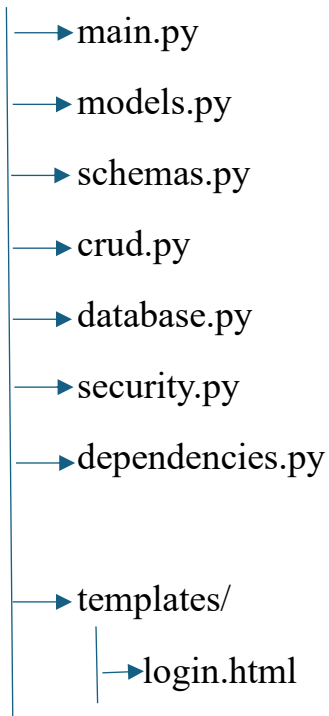
- To develop a centralized user management system.
- To maintain tenant information.
- To maintain property information.
- To establish relationships between users, tenants and properties.
- To implement secure authentication.
- To implement role-based authorization.
- To provide CRUD operations through REST APIs.
- To provide a simple web interface.
- To calculate user and tenant property counts.
- To retrieve related users and properties.

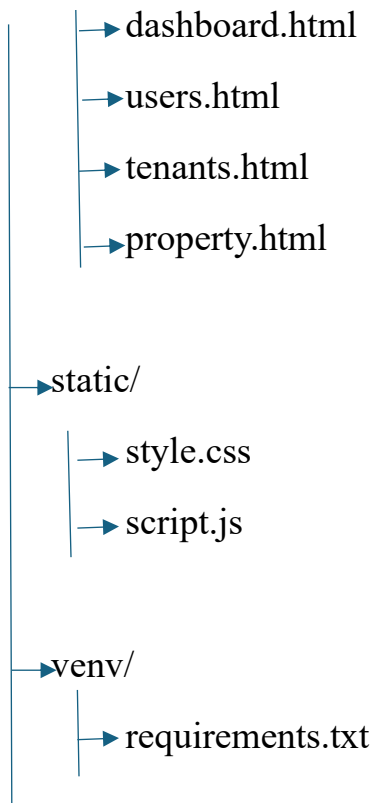
3. Technologies Used

Technology	Purpose
Python	Programming Language
FastAPI	API Framework
PostgreSQL	Database
SQLAlchemy	ORM (Object- Relational Mapping)
Pydantic	Data Validation
Uvicorn	ASGI Server (Asynchronous Server Gataway Interface)
Swagger UI	API Documentation
Postman	API Testing
Git	Version Control
GitHub	Source Code Repository
Pytest	Unit Testing
Jinja2	HTML Templates
Passlib	Password Hashing
Bcrypt	Password Encryption
Python-jose	JWT (Json Web Token) Authentication

4. Project Structure

USER SERVICE/





5. Features

5.1 Authentication:

- Login
- JWT Token Generation
- Password Hashing
- Protected APIs
- Logout

5.2 Dashboard

- Login
- Dashboard
- User Page
- Tenant Page

5.3 User Management:

- Create User
- View Users
- Update User
- Delete User
- Password Encryption
- Tenant Selection

5.4 Tenant Management

- Create Tenant
- View Tenant
- Update Tenant
- Delete Tenant

5.5 Property Management

- Create properties.
- View all properties.
- Update property information.
- Delete properties.

5.6 User–Tenant Relationship

- Each user belongs to a tenant.
- A tenant can have multiple users.
- The system displays the tenant associated with each user.

5.7 Tenant–Property Relationship

- Each property belongs to a tenant.
- A tenant can have multiple properties.
- The system can display the number of properties belonging to each tenant.

5.8 User–Property Relationship

- A user can have multiple properties.
- The system can display the number of properties belonging to each user.
- Users can view their associated properties.

5.9 Property Summary

- Display the total number of properties belonging to a user.
- Display the total number of properties belonging to a tenant.
- Display property details associated with a user or tenant.

5.10 User Count

- Display the number of users belonging to each tenant.
- Allow viewing the users associated with a tenant.

5.11 Database Relationships

- Tenant → Users: **One-to-Many**
- Tenant → Properties: **One-to-Many**
- User → Properties: **One-to-Many**
- Foreign keys maintain relationships between the tables.

5.12 REST API

- Uses FastAPI to provide REST API endpoints.
- Supports HTTP methods such as: GET, POST, PUT, DELETE

5.13 API Documentation

- FastAPI automatically provides Swagger UI.
- APIs can be tested directly through Swagger.
- Authentication can be tested using a JWT access token.

6. Python Packages Used

6.1 FastAPI

Install:

```
pip install fastapi
```

Purpose:

- Create APIs
- Route Requests
- Generate Swagger Documentation

Example:

```
app= FastAPI()
```

6.2 Uvicorn

Install:

```
pip install uvicorn
```

Purpose:

Runs FastAPI Application

Example:

```
uvicorn main:app --reload
```

6.3 SQLAlchemy

Install:

```
pip install sqlalchemy
```

Purpose:

- ORM (Object Relational Mapping)
- Communicates with PostgreSQL

Example:

```
db.query(User).all()
```

6.4 Psycopg2

Install:

`pip install psycopg2-binary`

Purpose:

PostgreSQL database driver

Example:

`Postgresql://username:mypassword@localhost/database`

6.5 Pydantic

Install:

`Pip install pydantic`

Purpose:

- Request Validation
- Response Validation

Example:

```
class UserCreate(BaseModel):  
    full_name:str
```

6.6 UUID (Universally Unique Identifier)

Purpose:

Generate Unique User IDs

Example:

```
import uuid  
uuid.uuid4()
```

6.7 Datetime

Purpose:

Manage Timestamps

Example:

```
from datetime import datetime
```

6.8 Pytest

Install:

```
pip install pytest
```

Purpose:

Unit testing

Example:

```
def test_home();  
aassert response.status_code == 200
```

6.9 FastAPI TestClient

Purpose:

Test APIs Without Postman

Example:

```
from fastapi.testclient import TestClient
```

6.10 Passlib

Hashes passwords

Example:

```
hash_password(password)
```

Purpose:

Never store plain Passwords

6.11 bcrypt

Hashing algorithm used by Passlib

Purpose:

Strong Password Encryption

6.12 python-jose

Creates JWT Tokens

Purpose:

Authentication

6.13 Jinja2

Loads HTML Pages

Example:

TemplateResponse()

Purpose:

Render HTML templates

7. Database Design**7.1 Tenant Table**

Schema: user_service.tenants

Fields:

- id
- tenant_name
- tenant_type
- created_by
- updated_by
- created_at
- updated_at

Relationship:

One Tenant



Many Users

7.2 User Table

Schema: user_service.users

Fields:

- id
- full_name
- phone_number
- email_id
- password
- role
- tenant_id
- address
- dob
- bio_details
- created_by
- updated_by
- created_at
- updated_at

Relationship:

Many Users



One Tenant

7.3 Property Table

Schema: property_services.properties

Fields:

- id
- tenant_id
- user_id
- property_code
- property_name
- property_location
- property_landmark
- latitude
- longitude
- area_size
- measurement_type
- plot_type
- ownership_type
- purchase_date
- market_value
- description
- verified_status
- risk_level
- fence_status
- is_active
- last_visited_at
- remarks
- visit_plan
- photo_url
- location_url
- embedded_url
- boundary_stones
- encroachment
- legal_disputes

- neighbour_issues
- vegetation_issues
- has_layout_plan
- has_patta_chitta
- latest_ec_date
- service_required
- additional_notes
- land_owner_name
- source
- created_by
- updated_by
- created_at
- updated_at

Relationship:

Property



tenant_id



Tenant



user_id



User

8. File Explanations

8.1 main.py

Purpose:

main.py is the main entry point of your FastAPI application. Think of it as the controller/router layer. It receives requests from the webpage or Swagger and decides:

Which API should handle this request?

For example:

POST /users

GET /users

GET /users/{user_id}

PUT /users/{user_id}

DELETE /users/{user_id}

Similarly, you have APIs for tenants and properties

Workflow:

Browser / Swagger



main.py



crud.py



Database

Example:

When the browser sends: GET /users

main.py receives the request and calls something similar to: `users = get_users(db)` from `crud.py`. So, `main.py` connects the API endpoints to your CRUD functions.

8.2 database.py

Purpose:

`database.py` is responsible for connecting FastAPI/SQLAlchemy to PostgreSQL.

Concept:

FastAPI



SQLAlchemy



`database.py`



PostgreSQL

It normally contains things such as: `engine`, `SessionLocal`, `Base`

Engine: The engine represents the connection configuration to PostgreSQL.

SessionLocal: Creates a database session.

For example: `db = SessionLocal()`

allows your CRUD functions to perform: `db.query(...)`, `db.add(...)`, `db.commit()`, `db.delete(...)`.

Base: Your models inherit from: `Base`

For example: `class User(Base):`

Therefore SQLAlchemy knows that `User` is a database model.

8.3 models.py

Purpose:

models.py defines the **database tables** using SQLAlchemy.

You have three main models:

- class Tenant(Base):
- class User(Base):
- class Property(Base):

These correspond to your PostgreSQL tables.

Tenant Model:

Tenant

|— id
|— tenant_name
|— tenant_type
|— created_by
|— updated_by
|— created_at
└— updated_at

It also contains:

- users = relationship(...)
- properties = relationship(...)

This means:

One Tenant

|— Many Users
└— Many Properties

User Model

User table contains information such as: id, phone_number, role, full_name, tenant_id, is_active, email_id, password, address, dob, bio_details, created_by, updated_by, created_at, updated_at. The important relationship is:

```
tenant_id = Column(
    UUID(as_uuid=True),
    ForeignKey("user_service.tenants.id")
).
```

This connects:

users.tenant_id



tenants.id

You also have:

```
tenant = relationship(
    "Tenant",
    back_populates="users"
).
```

This allows SQLAlchemy to understand the relationship between users and tenants.

Property Model

Property table contains all your property information. For example: property_code, property_name, property_location, latitude, longitude, area_size, measurement_type, market_value, ownership_type.

Most importantly: tenant_id & user_id.

These connect a property to both a tenant and a user.

Property

|— tenant_id → Tenant

|— user_id → User

Therefore: models.py describes how your database is structured.

8.4 schemas.py

Purpose:

schemas.py defines the data format going into and coming out of your API. It uses Pydantic. You have schemas such as:

- TenantCreate
- TenantResponse
- UserCreate
- UserResponse
- PropertyCreate
- PropertyResponse
- PropertyUpdate
- UserPropertySummary
- TenantPropertySummary

Create vs Response

This distinction is very important.

For example: class UserCreate(BaseModel):

defines what the API accepts when creating a user.

It contains: full_name, phone_number, email_id, password, role, tenant_name,..

But: class UserResponse(BaseModel):

defines what the API sends back. Notice that the password is not included.

That's important because:

Password
↓
Database
↓
Hashed password

but:

API Response
↓
NO password

So your password can be stored securely without exposing it through your webpage/API response.

8.5 crud.py

Purpose

crud.py contains our actual database operations.

CRUD means:

- C → Create
- R → Read
- U → Update
- D → Delete

crud.py handles all three major entities:

- Tenant
- User
- Property

Tenant CRUD

We have functions such as:

- `create_tenant()`
- `get_tenants()`
- `get_tenant()`
- `update_tenant()`
- `delete_tenant()`

These perform operations on the Tenant table.

For example:

```
def get_tenants(db):
```

```
    return db.query(Tenant).all()
```

means: Get all tenants from PostgreSQL.

User CRUD

We have:

- `create_user()`
- `get_users()`
- `get_user()`
- `update_user()`
- `delete_user()`
- `authenticate_user()`

These work with the User table. Our `create_user()` also performs an important operation:

User creation

↓

Find tenant

↓

Check tenant exists



Hash password



Create user



Save user

Property CRUD

We have:

- `create_property()`
- `get_properties()`
- `get_property()`
- `update_property()`
- `delete_property()`

These work with the Property table. Our `create_property()` checks:

Does tenant exist?



Does user exist?



Create property

This prevents a property from being created with an invalid tenant or user.

Summary functions

Our `crud.py` also contains functions for relationships.

For example: `get_user_property_summary()`

answers:

Which tenant does the user belong to?

How many properties does the user have?

Which properties belong to the user?

And:

`get_tenant_property_summary()`

answers:

How many properties does the tenant have?

Which properties belong to the tenant?

So:

`crud.py` is responsible for the application's database logic.

8.6 security.py

Purpose:

`security.py` handles password security and JWT token creation.

Project uses:

- `bcrypt`
- `JWT`
- `HS256`

Password hashing

We have: `hash_password(password)`

Workflow

User enters:

mypassword123



bcrypt



\$2b\$12\$.....



PostgreSQL

We never need to store: “mypassword123” as plain text.

Password verification

We also have: `verify_password()`

During login:

Entered password



`verify_password()`



Stored bcrypt hash



Match?

If it matches, login succeeds.

JWT

We also have: `create_access_token()`

This creates the JWT token after successful login.

Workflow:

Email + Password



Authentication



JWT created



Browser stores token



Token sent with API requests

8.7 dependencies.py

Purpose:

`dependencies.py` handles authentication and authorization dependencies. Our important function is: “`get_current_user()`”. It checks whether the request contains a valid JWT.

Workflow:

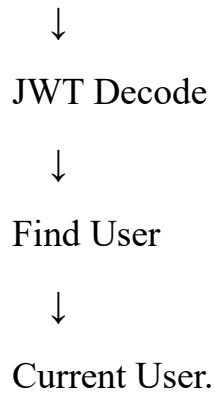
API Request



Authorization Header



Bearer Token



8.8 templates folder

Contains all HTML Pages

8.8.1 Login.html

Purpose:

This is your login webpage. The user enters: Email & Password.

Then JavaScript sends: POST /login, The server checks the credentials.

If successful:

JWT Token

↓

localStorage

↓

Dashboard

8.8.2 dashboard.html

Purpose:

This is the main page after login. It acts as the navigation page for your system.

For example:

Dashboard

└─ Users

└─ Tenants

└─ Properties

└─ Logout

Instead of manually typing API URLs, the user can navigate through the web interface.

8.8.3 users.html

Purpose:

This is my User Management UI. It allows you to perform user CRUD operations.

- Create User
- View Users
- Edit User
- Delete User

You also have functionality to view a user's properties.

Example:

Goku

Property Count: 3

View Properties

↓

Property 1

Property 2

Property 3

8.8.4 tenants.html

Purpose:

This is your Tenant Management UI.

For example:

Z Company

GROUP

5 Properties

3 Users

You also have buttons for:

- Properties
- Users
- Edit
- Delete

So the tenant page provides a quick overview of the tenant.

8.8.5 properties.html

Purpose

This is your Property Management UI.

It allows us to:

- Create Property
- View Properties
- Edit Property
- Delete Property

8.9 static folder

Contains: CSS, JS, Images

8.9.1 style.css

Responsible for: Colors, Buttons, Layout, Tables, Fonts

8.9.2 script.css

Responsible for: Calling APIs, Showing Tables, Updating Data, Deleting Data, Login, Logout.

9. API Endpoints

9.1 Login

POST /login

Purpose: Generate JWT Token

9.2 User APIs

9.2.1 Create User: POST /users

Purpose: Create a New User

9.2.2 Get All Users: GET /users

Purpose: Retrieve all users.

9.2.3 Get User By ID: GET /users/{user_id}

Purpose: Retrieve Specific User.

9.2.4 Update User: PUT /users/{user_id}

Purpose: Update User Details

9.2.5 Delete User: DELETE /users/{user_id}

Purpose: Delete User.

9.2.6 User Property Summary: GET /users/{user_id}/properties

Purpose: Returns the user's tenant and properties.

9.3 Tenant APIs

9.3.1 Create Tenant: POST /tenants

Purpose: Creates a new tenant.

9.3.2 Get All Tenant: GET /tenants

Purpose: Returns all tenants.

9.3.3 Get User By ID: GET /tenants/{tenant_id}

Purpose: Returns information about a particular tenant.

9.3.4 Update Tenant: PUT /tenants/{id}

Purpose: Updates tenant information.

9.3.5 Delete Tenant: DELETE /tenants/{id}

Purpose: Deletes a tenant.

9.3.6 Tenant Property Summary: GET /tenants/{tenant_id}/properties

Purpose: Returns all properties belonging to a tenant.

9.4 Property APIs

9.4.1 Create Property: POST /properties

Purpose: Creates a property and associates it with both: Tenant & User

9.4.2 Get All Properties: GET /properties

Purpose: Returns all properties.

9.4.3 Get Property by ID: GET /properties/{property_id}

Purpose: Returns one property.

9.4.4 Update Property: PUT /properties/{property_id}

Purpose: Updates property information.

9.4.5 Delete Property: DELETE /properties/{property_id}

Purpose: Deletes a property.

10. Figures

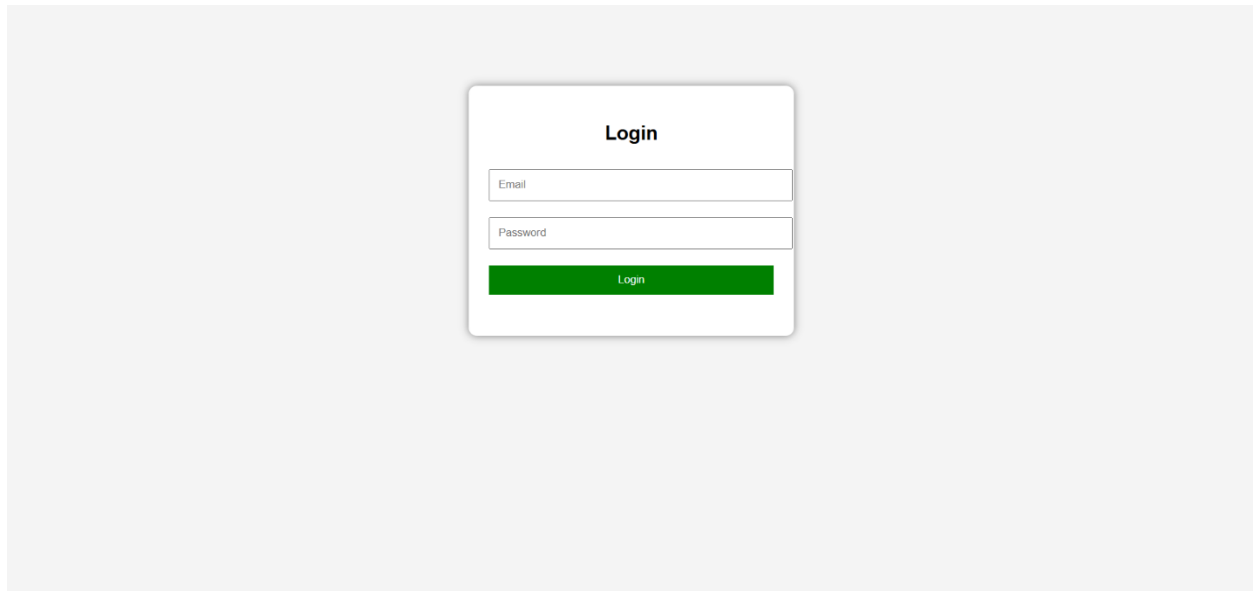


Figure 10.1 Login page

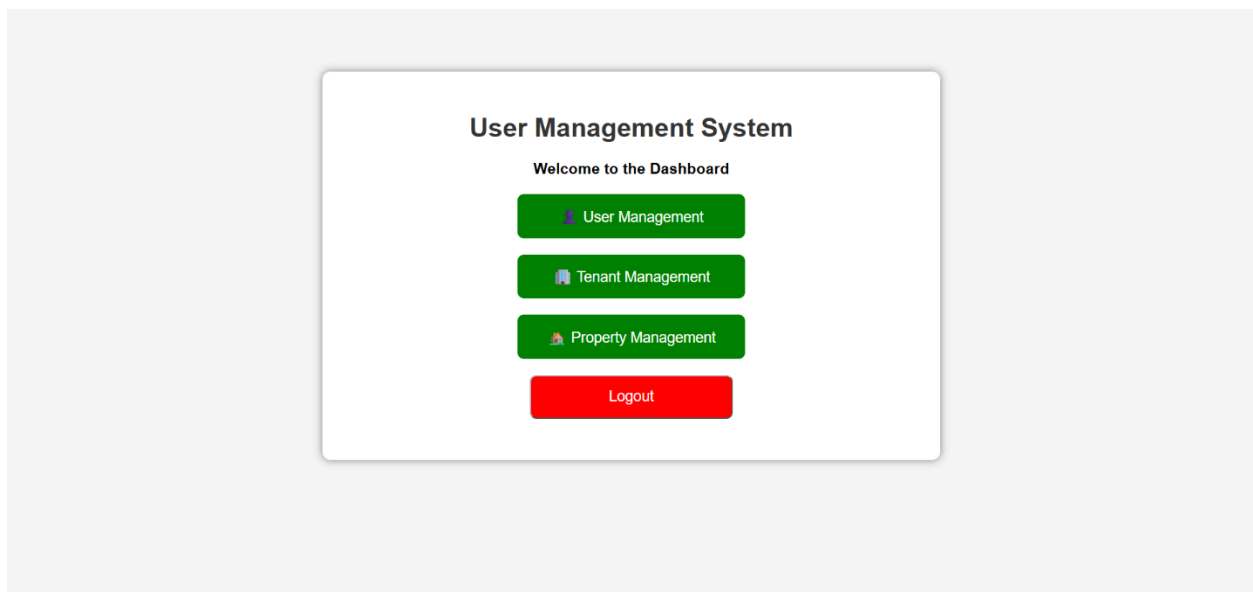


Figure 10.2 Dashboard

TOP BUTTONS

Dashboard

Logout

Create User

Full Name

Enter full name

Phone Number

Example: +919876543210

Email

Enter email

Password

Enter password

Password is required when creating a user. Leave it empty when editing to keep the existing password.

Role

ADMIN / MANAGER / USER

Tenant Name

Enter tenant name

Address

Address

Enter address

Date of Birth

mm / dd / yyyy



Bio Details

Enter bio details

Created By

Created by

Updated By

Updated by

Account Status

Active



Create User

All Users												
Full Name	Phone	Email	Role	Tenant	Properties	Address	Date of Birth	Bio	Created By	Updated By	Status	Actions
Goku	+917896541230	Goku@gmail.com	Web Design	Dog Company	Chennai	2000-07-07	Male	System	Admin	Active		View Properties Edit Delete
Ravi	+919874563210	Ravi@gmail.com	Developer	Dog Company	Salem	1998-02-07	Male	System	Admin	Active		View Properties Edit Delete

Figure 10.3 Users Page

Dashboard

Logout

Tenant Management

Create Tenant

Tenant Name

Individual

Created By

Create Tenant

Tenant List

Refresh

ID	Name	Type	Property Count	User Count	Created By	Created At	Actions
08264b9e-02ae-451a-96ae-bad8a0c8a396	Z Company	GROUP	0	0	Z Company	2026-06-26T18:27:49.069358	Properties Users Edit Delete
8a4a905d-c603-4c12-8304-7019ebb16eaa	Cat Company	GROUP	0	0	Cat Company	2026-06-27T09:49:01.351181	Properties Users Edit Delete
90ef9406-fbd8-4e51-a11f-a041251dc181	Dog Company	ENTERPRISE	0	2	System	2026-07-03T11:35:38.559032	Properties Users Edit Delete

Figure 10.4 Tenant Page

Dashboard

Users

Logout

Property Management

Create Property

Tenant

Select Tenant

Select Tenant

Z Company

Cat Company

Dog Company

Property Code

Latitude

Area Size

Plot Type

Select

Purchase Date

mm/dd/yyyy

User

Select User

Property Name

Property Landmark

Longitude

Measurement Type

Select

Ownership Type

Select

Market Value

Dashboard

Users

Logout

Property Management

Create Property

Tenant

Select Tenant

Property Code

Property Location

Latitude

Area Size

Plot Type

Select

Purchase Date

mm/dd/yyyy

User

Select User

Select User

Goku - Dog Company

Ravi - Dog Company

Property Landmark

Longitude

Measurement Type

Select

Ownership Type

Select

Market Value

Verified Status Select	Risk Level Select
Fence Status No Fence	Is Active Active
Boundary Stones 	Encroachment No
Latest EC Date mm / dd / yyyy	Land Owner Name
Source 	Created By
Updated By 	
Description 	
Remarks 	

Visit Plan
Legal Disputes
Neighbour Issues
Vegetation Issues
Service Required
Additional Notes

Last Visited At mm / dd / yyyy -- : -- --	Photo URL
Location URL 	Embedded URL

Figure 10.5 Property Page

11. Git Commands Used

11.1 Initialize:

```
git init
```

11.2 Add Files:

```
git add .
```

11.3 Commit:

```
git commit -m "Give a Message"
```

11.4 Connect GitHub:

```
git remote add origin Repository_URL
```

11.5 Push:

```
git push -u origin main
```

13. Conclusion

The User, Tenant and Property Management System successfully provides a centralized platform for managing tenants, users, and properties. The system uses FastAPI for backend API development, PostgreSQL for reliable data storage, and SQLAlchemy for database interaction. The system implements CRUD operations for tenants, users, and properties, allowing users to create, view, update, and delete records. The relationships between tenants, users, and properties make it possible to identify which tenant a user belongs to, how many users and properties a tenant has, and which properties belong to a particular user. The project also includes JWT-based authentication, role-based authorization, and bcrypt password hashing. Passwords are stored securely as hashes and are not exposed through the web interface or API responses. The web interface provides separate management pages for Users, Tenants, and Properties, making the system easier to use and providing useful information such as user counts and property counts. Overall, the project demonstrates how a real-world management system can combine backend APIs, database relationships, authentication, security, and a web interface into one complete application.

